Results on PBM-driven ODE discovery and ODE parameter estimation

Urh Primožič, IJS

ODE discovery with ProBMoT

- processes between entities (predator–prey models, ...)
- build a list of possible models $[\vec{x}'(t) = \vec{F}_i(\vec{x}(t), t, \theta^{(i)})]_i$
- **estimate model parameters** $\theta^{(i)}$
- evaluate all possible models

Structure changes

- migration from Java to Python
- gradient opt instead of EA
- orders of magnitude faster parameter estimation
- support for conditional statements

$$x'(t) = \begin{cases} f(x,t,\theta) & c(x,t,\theta) \geq 0 \\ g(x,t,\theta) & \text{otherwise} \end{cases}$$

New syntax

```
predator = Entity(Var("size"))
prey = Entity(Var("size"))
x, y = prey["size"], predator["size"]
r = Const("growth_rate")
K = Const("K")
...

exponential = Process(Ode(x, r * x))
logistic = Process(Ode(x, r * x * (1 - x / K)))
growth = ChooseProcess(exponential, logistic, name="growth")

predation = Process(Ode(x, -a * x * y), Ode(y, b * x * y - m * y),
                    name="predation")

models = Model(growth, predation).induce()
add_data(data)
results = estimate_models_collocation(models, t)
```

How this would look in ProBMoT

```
library LV;  // "inside file LV.pbl"
template entity Population { vars: size {aggregation: sum}; }
template process Growth(p: Population) {}
template process Exponential: Growth {
  consts: r {range: <0, 5>};
  equations: td(p.size) = r * p.size; }
template process Logistic: Growth {
  consts: r {range: <0, 5>}, K {range: <0, 100>};
  equations: td(p.size) = r * p.size * (1 - p.size / K); }
template process Predation(x: Population, y: Population) {
  consts: a {range: <0, 5>}, b {range: <0, 5>}, m {range: <0, 5>};
  equations: td(x.size) = -a * x.size * y.size,
             td(y.size) = b * x.size * y.size - m * y.size; }
```

```
incomplete model LVModel : LV;  // "LVModel.pbm, + a task .xml"
entity prey : Population { vars: size {role: endogenous; initial: null}; }
entity pred : Population { vars: size {role: endogenous; initial: null}; }
process growth(prey) : Growth {}
process predation(prey, pred) : Predation {}
```

Templates

```
def temp_lim(var, env, name):
    """Temperature limitation of any rate: none, linear or exponential."""
    T = env["temperature"]
    refTemp = Const("refTemp", range=(5, 10))
    theta = Const("theta", range=(1.06, 1.13))
    return ChooseProcess(
        Process(Algebraic(var, 1), name="none"),
        Process(Algebraic(var, T / refTemp), name="linear"),
        Process(Algebraic(var, theta ** (T - refTemp)), name="exponential"),
        name=name)

influences = [temp_lim(phyto[f"temp{r}Lim"], env, name=f"temp{r}Influence")
              for r in ["Growth", "Resp", "Mort", "Exc", "Sed"]]
```

Templates in ProBMoT

```
// Zurich library: one of five copies (Growth, Resp, Mort, Exc, Sed)
template process TempMortInfluence(pop : Population, env : Environment) {}
template process NoTempMortLim : TempMortInfluence {
  equations: pop.tempRespLim = 1; }   // tempRespLim, not tempMortLim
template process TempMortLim : TempMortInfluence {
  consts: refTemp {range: <5,10>}, minTemp {range: <0, 6>},
          optTemp {range: <5, 12>}; }
template process Linear1TempMortLim : TempMortLim {
  equations: pop.tempRespLim = env.temperature/refTemp; }
template process ExponentialTempMortLim : TempMortLim {
  consts: theta {range: <1.06, 1.13>};
  equations: pop.tempRespLim = pow(theta, env.temperature - refTemp); }
```

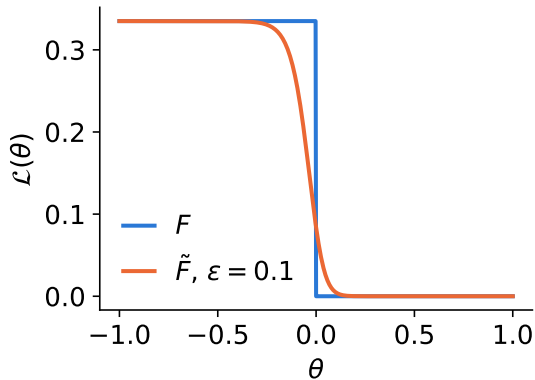
- empty templates `{}` only group alternatives (14 in the library)
- the same family five times, ≈ 45 lines each
- constants of the parent are fitted in every child, used or not
- copy–paste: the whole `TempMort` family sets `tempRespLim`

Relaxed conditionals

$$F = \begin{cases} f & \theta \geq 0 \\ g & \text{otherwise} \end{cases}$$

$$\tilde{F} = \frac{f+g}{2} + \frac{f-g}{2} \tanh \frac{c}{\varepsilon}$$

plot: $f = 1, g = 0$



TODOs

- JIT-compiled $F(x, t, \theta)$
- clever structure search
- precise estimation only for the best candidates

Parameter estimation

- $x\colon [t_0, t_1] \to \mathbb{R}^n$
- $x_{\text{obs}} = (x_{o_1}, x_{o_2}, \ldots, x_{o_{n_{\text{obs}}}}) \in \mathbb{R}^{n_{\text{obs}}}$

Given

- ODE $x'(t) = F(x(t), t, \theta)$
- measurements $(t_i, x_{\text{obs}}(t_i) + \epsilon_i)$, $\epsilon_i \sim N(0, \sigma^2)$

estimate θ.

Integration using multiple shooting[1]

- integration: pick $x(t_0)$, solve ODE for $x_\theta(t)$, min discrepancy $\mathcal{L}(x_\theta)$ on data
- problem: chaotic systems
- solution: integrate shorter intervals, join trajectories

[1] Ozgur Aydogmus and Ali Hakan Tor (2020). *A Modified Multiple Shooting Algorithm for Parameter Estimation in ODEs Using Adjoint Sensitivity Analysis*. arXiv: 2006.15740 [math.OC].

Integration using multiple shooting[2]

Multiple shooting

find θ, s_i minimizing discrepancy on data

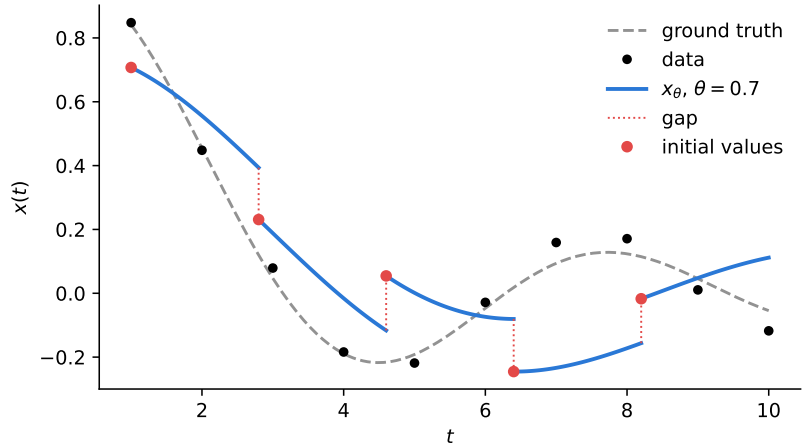
$$\sum_i \mathcal{L}(x_i), \quad \text{where } x_i' = F(x_i, t, \theta), x_i(t_i) = s_i$$

subject to

$$x_i(t_{i+1}) = s_{i+1} \quad \forall i = 0, 1, \ldots, N_{\text{subintervals}} - 2$$

[2] Ozgur Aydogmus and Ali Hakan Tor (2020). *A Modified Multiple Shooting Algorithm for Parameter Estimation in ODEs Using Adjoint Sensitivity Analysis.* arXiv: 2006.15740 [math.OC].

Integration using multiple shooting



Multiple shooting for $x(t) = \sin(\theta t)/t$ on $[1, 5]$, $\theta_{\text{true}} = 1$, evaluated at $\theta = 0.7$ with 10 intervals. Data: 16 points with noise $\mathcal{N}(0, 0.05^2)$; initial values perturbed by $\mathcal{U}(-0.1, 0.1)$.

Gradient matching for fully-observed systems

- $\hat{x}(t)$ - interpolant of the data
- interpolation: Gaussian processes[3], cubic splines[4]

Least squares formulation

Find $\operatorname{argmin}_\theta \|\hat{x}'(t) - F(\hat{x}(t), t, \theta)\|$

[3] Philippe Wenk et al. (2019). *Fast Gaussian Process Based Gradient Matching for Parameter Identification in Systems of Nonlinear ODEs*. arXiv: 1804.04378 [stat.ML].

[4] J. M. Varah (1982). "A Spline Least Squares Method for Numerical Parameter Estimation in Differential Equations". In: *SIAM Journal on Scientific and Statistical Computing* 3.1, pp. 28–46. DOI: 10.1137/0903003. eprint: https://doi.org/10.1137/0903003.

Integral matching[5]

Derivatives are noisy $\propto \frac{1}{|t_{i+1}-t_i|}$!

Minimize

$$\int_{t_0+\Delta}^{t_1} \left\| \hat{x}(t) - \hat{x}(t-\Delta) - \int_{t-\Delta}^{t} F(\hat{x}(\tau), \tau, \theta)\, d\tau \right\| dt$$

[5] Baptiste T Rossi (2025). *ODE Parameter Identification: An Integral Matching Approach.*

Collocation[6]

- $x\colon I \to \mathbb{R}^n$ partially observed
- choose l *collocation* points
  $(\tau_1, s_1), (\tau_2, s_2), \ldots, (\tau_l, s_l) \in I \times \mathbb{R}^n$
- $\hat{x}(t)$: interpolant of $(\tau_i, s_i)_i$, hyperparameters ϕ
- choose $s = (s_1, \ldots, s_l)$ and ϕ subject to $\hat{x}(t)$ solves the ODE and matches the data

[6] Samuel W. K. Wong et al. (2023). *MAGI: A Package for Inference of Dynamic Systems from Noisy and Sparse Data via Manifold-constrained Gaussian Processes*. arXiv: 2203.06066 [stat.CO]; S Vasantharajan et al. (1988). "Simultaneous strategies for optimization of differential-algebraic systems". In: DOI: 10.1184/R1/6467471.

Collocation

Let $\hat{x}(t)$ be the interpolant of (τ_i, s_i) with hyperparameters ϕ and collocation points $s = (s_1, \ldots, s_l)$. Let (t_i, y_i) be a set of noisy measurements.

Find s, θ, ϕ minimizing[a]

$$\sum_i \|\hat{x}(t_i) - y_i\|$$

subject to[b]

$$\hat{x}'(t) = F(\hat{x}(t), t, \theta).$$

[a] Wong et al. (2023) use a Bayesian approach.

[b] Gradient matching. Integral matching can be used as well.

Collocation vs integration

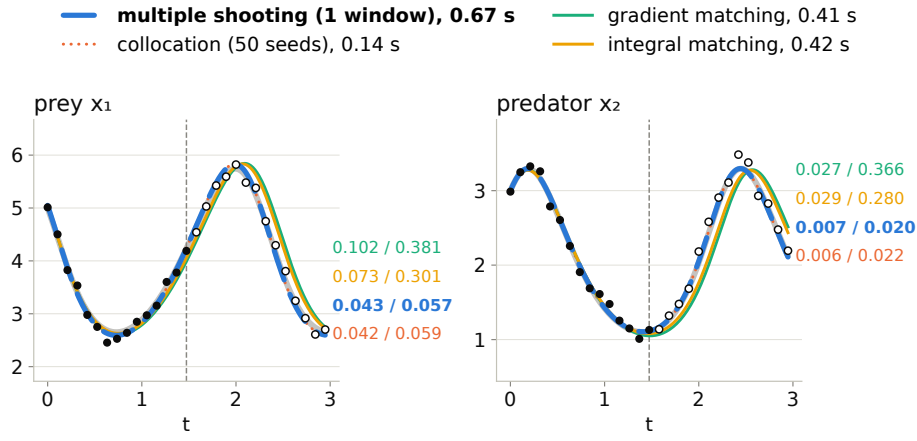
multiple shooting	collocation
Freezes θ, solves the ODE	Changes both θ and s until the trajectory satisfies the ODE
Additional parameters are initial values	Additional parameters are collocation points of the interpolant

Design choices and challenges

- Bayesian, least squares, *constrained problem*
- two-step approach vs joint estimation
- gradient vs integral matching, integration interval
- kernel choice; collocation, manifold and discrepancy points
- chaotic loss surface: **homotopy approaches**[7]

[7] Chandrika Vyasarayani et al. (Mar. 2012). "Single-shooting homotopy method for parameter identification in dynamical systems". In: *Physical Review E* 85, p. 036201. DOI: 10.1103/PhysRevE.85.036201.

Lotka–Volterra

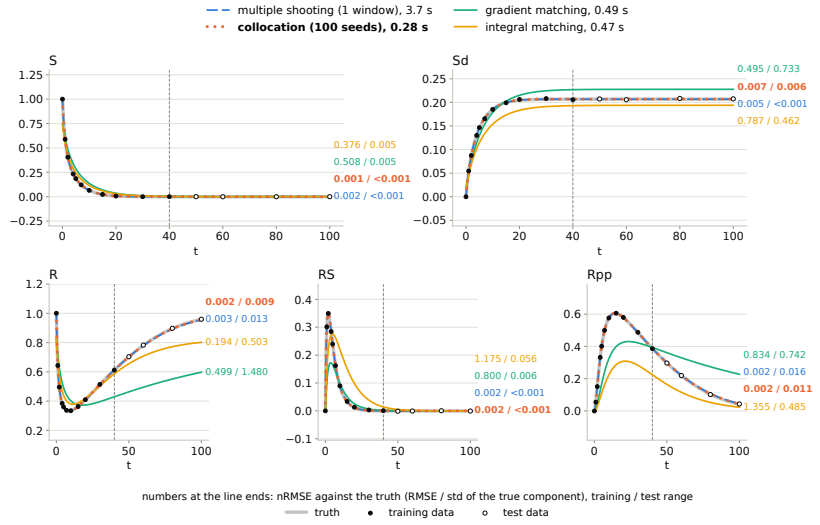


numbers at the line ends: nRMSE against the truth (RMSE / std of the true component), training / test range

truth training data test data

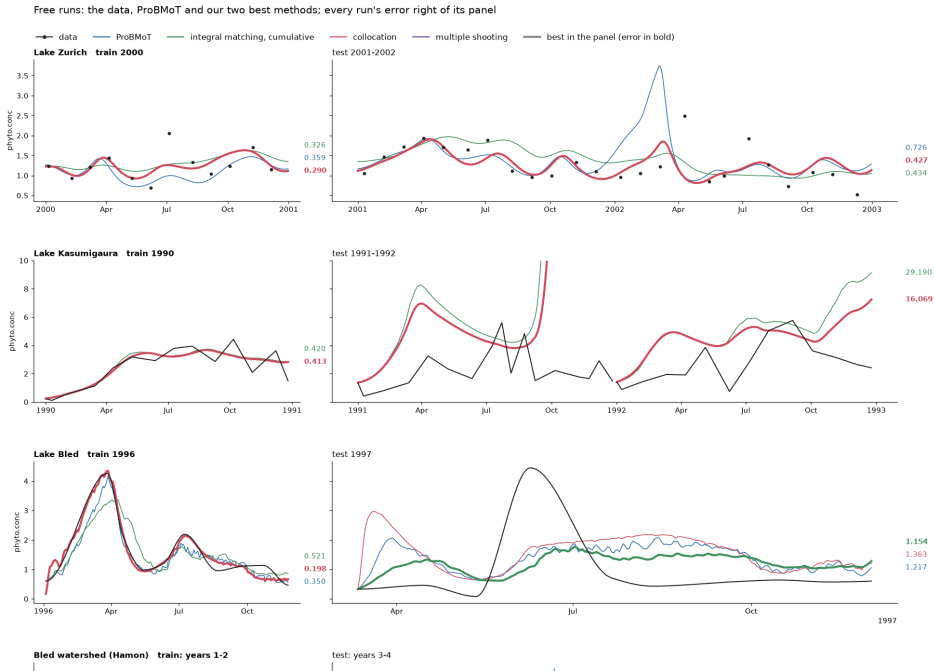
$$x_1' = \theta_1 x_1 - \theta_2 x_1 x_2, \quad x_2' = -\theta_3 x_2 + \theta_4 x_1 x_2$$

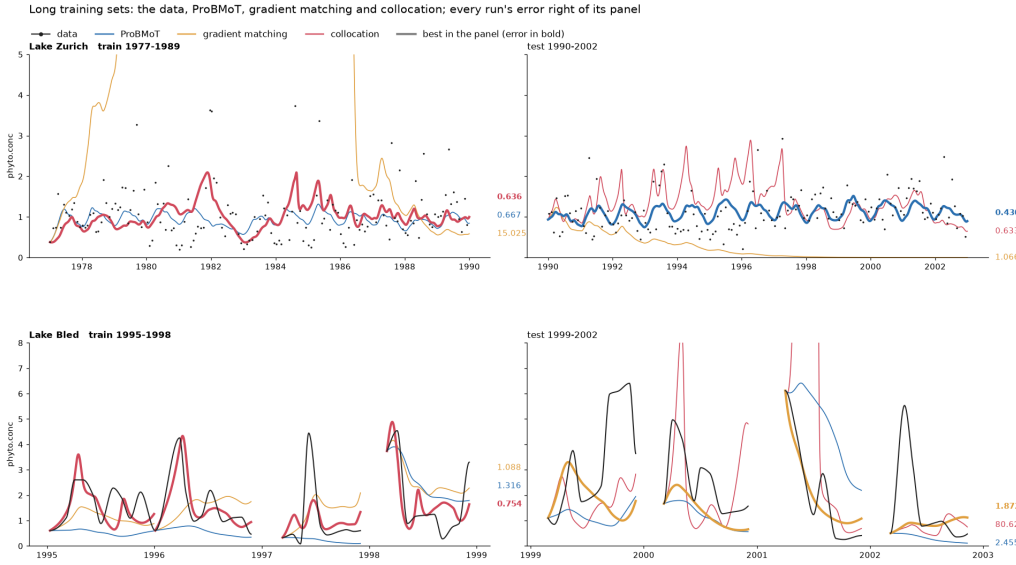
Protein transduction



$$S' = -\theta_1 S - \theta_2 SR + \theta_3 R_S, S'_d = \theta_1 S, R' = -\theta_2 SR + \theta_3 R_S + \theta_5 \frac{R_{pp}}{\theta_6 + R_{pp}}$$

$$R'_S = \theta_2 SR - \theta_3 R_S - \theta_4 R_S, R'_{pp} = \theta_4 R_S - \theta_5 \frac{R_{pp}}{\theta_6 + R_{pp}}$$


Free runs: the data, ProBMoT and our two best methods; every run's error right of its panel
data
ProBMoT
integral matching, cumulative
collocation
multiple shooting
best in the panel (error in bold)
Lake Zurich train 2000
test 2001-2002
phyto.conc
0.326
0.359
0.290
0.726
0.427
0.434
Lake Kasumigaura train 1990
test 1991-1992
0.420
0.413
29.190
16.069
Lake Bled train 1996
test 1997
0.521
0.198
0.350
1.154
1.363
1.217
Bled watershed (Hamon) train: years 1-2
test: years 3-4



Errors: RMSE of phyto.conc in free runs (lower is better); every method's structure is the one with its lowest training error. ProBMoT: its task's settings (DE, CVODE at 1e-3), simulated here by PyBM.
Zurich: one free run over the training years and one over the test years, each from the data on its first day; RMSE on the measured days (12 a year). Collocation: weekly seeds, the ODE as the week's integral.
Bled: the seasons are separate (no data over the winters): every season is a run from the data on its first day. Collocation: daily seeds, no ODE across a winter.
ProBMoT on Bled: only on collocation's structure; it fits one initial concentration to all four seasons.

Research directions

- small, domain-specific benchmarks, just integration methods
- 2026: 1100 synthetic benchmark problems - no methods
- **no good benchmark**
- collocation seems to join different approaches together $\rightarrow$ better method?



github.com/urhprimozic/PyBM



github.com/urhprimozic/odestimation